

第21章 异常的处理

21.1 错误和异常

错误和异常处理是编程中的重要机制，用于处理程序运行时可能出现的意外情况。通过合理的错误和异常处理，可以提高程序的健壮性、可维护性和用户体验。尽管错误和异常都涉及程序执行中的问题，但二者存在区别。错误通常指严重的、不可恢复的问题，如内存耗尽、系统崩溃等，这些问题一般不由应用程序直接处理，而是交由运行时环境或操作系统管理。相比之下，异常是指程序运行时可预见且、可恢复的问题，如文件未找到、除零错误或无效输入等，这类问题通常需要程序主动捕获并处理。

程序应尽可能避免执行过程中出现错误，但若错误不可避免地发生，程序员往往也难以找到有效的处理方法。相比之下，异常是程序设计中可以预见的执行错误，虽然无法完全避免，但我们可以在编写程序时就做好应对准备。因此，我们真正需要关注和处理的是这些异常情况。由于历史原因，错误和异常这两个术语经常被混用，但实际上它们主要指代的就是这里讨论的异常情况。在接下来的讨论中，我们将不再对二者进行区分。

异常总是在函数执行过程中产生的，其根本原因可以归结为以下三种情况：

- (1) 函数执行后，不满足其后置条件。如，函数返回了一个无效对象。
- (2) 函数执行过程中，调用了不满足前置条件的其他函数。
- (3) 实例化的对象不满足类的不变式。如，无效的数据成员。

一个理想的函数应当能够处理各种逻辑分支，包括上述的异常情况。然而，在面对异常分支时，这一目标往往难以实现：一方面，分支条件可能复杂且难以描述；另一方面，即便确定了这些条件，也可能难以进行有效的处理。例如，对于语句 `A * pA = new A(2);`，开发者通常只考虑正常情况，但如果内存不足导致 `new A(2)` 失败，或者在构造 `A` 类对象时出现问题，该如何妥善处理呢？

21.2 传统的处理方式

一个程序能够编译通过，不代表程序在执行过程中不会产生运行时错误或异常。处理这些运行时错误的常见方式有：

21.2.1 直接终止

出现异常时，程序不做任何处理，直接终止程序。例如：

```

1  #include <iostream>
2  int divide(int a, int b) {
3      return a / b;
4  }
5  int main( ) {
6      std::cout << "Result =" << divide(10, 0) << std::endl;
7  }

```

当除零发生时，程序无法继续执行，由运行环境终止程序。

21.2.2 使用断言

在程序中使用静态断言(static_assert)可以在编译期检测错误。运行时断言(assert)也称断言宏，相当于使用条件编译，只在调试模式(Debug)下参与编译，在发行模式(Release)下不参与编译。在 debug 模式下，编译生成的二进制可执行代码中包含检查断言的执行代码，便于在调试期给出更明确的错误信息并终止程序。例如：

```

1  #include <iostream>
2  #include <cassert>
3  static_assert(sizeof( void * ) == 4, "只能运行在 32 位平台");
4  class A {
5  public:
6      A( int n ) :mNum( n ) {}
7      operator int( ) const { return mNum; }
8  private:
9      int mNum = 10;
10 };
11 int divide( const A & a, int b ) {
12     static_assert( std::is_convertible<A, int>::value, "参数类型 A 不能转成整数");
13     assert( b != 0 );
14     return static_cast<int>(a) / b;
15 }
16 int main( ) {
17     A a( 100 );
18     std::cout << "Result =" << divide( a, 0 ) << std::endl;
19 }

```

21.2.3 使用条件检查

程序中也可以使用条件分支，明确地区分异常产生条件，并给出相应处理。例如：

```

1  #include <iostream>
2  int divide( int a, int b ) {

```

```

3     if (b == 0) {
4         std::cout << "除数不能为零！" << std::endl;
5         return a >= 0 ? INT_MAX : INT_MIN;
6     }
7     return a / b;
8 }
9 int main( ) {
10     std::cout << "Result =" << divide( 10, 0 ) << std::endl;
11 }

```

这种方式的问题在于，处理错误的代码与程序的正常功能代码混合在一起了，不利于代码的理解和维护。

21.2.4 使用错误码

另一种处理方式是使用错误码。当程序进入异常逻辑分支时，可以直接返回错误码，告知调用者函数未能成功执行，而后续的处理则由调用者负责。例如：

```

1  #include <iostream>
2  using namespace std;
3  enum class FileError {
4      Success = 0,
5      InvalidFilename = 1,
6      FileNotFound = 2,
7      PermissionDenied = 3
8  };
9  FileError openFile(const std::string & filename) {
10     if (filename.empty( )) {
11         return FileError::InvalidFilename;
12     }
13     // 打开文件的操作
14     return FileError::Success;
15 }
16 int main( ) {
17     FileError result = openFile("");
18     if (result != FileError::Success) {
19         cout << "打开文件失败！ 错误码：" << static_cast<int>(result) << endl;
20     }
21     return 0;
22 }

```

在返回错误码时，通常使用 0 表示函数执行成功，非 0 值则用于表示不同的错误类型。特别地，在一些应用场景中，常用非负整数表示正常的结果，用负整数表示错误类型。

21.2.5 比较

在上述几种处理方式中，直接终止是最不理想的，程序设计时应尽量避免；使用断言虽然有所改进，但仅限于通知错误，无法进行有效的后续处理；条件检查会使正常逻辑与异常处理逻辑耦合，增加维护难度；而错误码虽然相对较好，但也存在局限性，并非适用于所有场景。

错误码的局限性主要体现在以下几点：

- (1) 错误码通常为整型值，这在需要返回其他类型的结果时显得不够灵活。
- (2) 在复杂或难以描述的异常情况下，准确判断并设置合适的分支条件颇具挑战，可能导致代码逻辑变得繁琐且难以维护。
- (3) 错误码携带的信息量有限，给后续处理带来了不便。

针对这些局限性，可以考虑以下方案：对于第一类问题，可以利用 C++ 新标准中的 `std::optional` 和 `std::expected` 来改进设计与实现；对于第二类问题，可以引入异常处理机制，以更好地应对复杂场景；对于第三类问题，则可以根据具体情况选择使用异常机制或 `std::expected`。

接下来我们将先讨论异常处理机制，在本章的最后介绍 `std::optional` 和 `std::expected`。

21.3 异常

当程序无法继续执行或者不满足指定条件时，通常有两种常见的处理方式：一种是通过返回错误码来通知调用者产生了异常；另一种则是采用异常处理机制，允许程序在遇到异常时抛出异常并由上层代码捕获和处理。

采用异常处理机制时，虽然一般面向对象程序设计语言通常采用异常对象表示异常，但在 C++ 中异常可以用任意类型的数据表示，从整型、指针到对象都可以。例如：

```
1  throw 1;                //抛出整型异常，值为 1
2  throw "出现异常";       //抛出字符串类型的异常，内容为"出现异常"
3  throw MyE( );           //抛出 MyE 类型的异常对象
4  string("abc").substr(10); //抛出 std::length_error 异常对象
```

21.3.1 异常的抛出

异常可以通过 `throw` 语句手动抛出，也可以在执行语句的过程中自动抛出。例如：

```
1  void func( int param ) {
2      if(param > 100 ) {
3          throw "参数过大"; //手动抛出异常
4      }else {
5          //若分配空间失败，自动抛出 std::bad_alloc 类型的异常
```

```

6      A * pA = new A( param );
7      pA->doSome( ); //也可能抛出异常
8      delete pA;
9  }
10 }
```

若需要手动抛出异常，可以使用 `throw` 表达式，语法格式为：

格式 1: `throw 表达式;`

格式 2: `throw;`

形式 1 中的表达式表示一个异常，其类型可以是任意类型，但最常见的还是 `std::exception` 类的派生类对象。形式 2 是形式 1 的一种特殊情况，它表示在处理异常的过程中，将当前捕获的异常继续向上层抛出。请参见异常的再抛出。

21.3.2 C++标准异常类

在 C++标准库中，有两个与异常处理相关的头文件：`<exception>` 和 `<stdexcept>`。其中，`<exception>` 定义了所有异常类的基类 `std::exception` 及其相关函数，而 `<stdexcept>` 则预定义了一系列常用的派生异常类，这些派生类都继承自 `std::exception`。以下是 C++11 中部分标准异常类的分类：

1. 与 C++语言特有语法相关的异常：

- (1) `bad_alloc`: `new` 表达式分配空间失败时抛出的异常。
- (2) `bad_cast`: 执行类型转换操作失败抛出的异常。
- (3) `bad_typeid`: `typeid` 的参数为零或空指针时抛出的异常
- (4) `bad_exception`: 非预期的特殊异常

2. 程序逻辑错误导致的异常，这类异常派生自 `std::logic_error` 类。这类错误是由于程序员的过错而产生的，理论上，在执行前，通过静态测试、代码分析等静态工具和方法是可以检测到的。

- (1) `domain_error`: 定义域错误，用于第三方类库
- (2) `invalid_argument`: 无效实参
- (3) `length_error`: 超过了允许的最大值
- (4) `out_of_range`: 访问不在定义范围内的元素

3. 程序运行时产生的异常，这类异常派生自 `std::runtime_error`。这类错误通常是由于硬件或系统的原因而产生的，程序员无法控制且不易避免。此类错误只能在程序执行时才能检测

到。

- (1) `range_error`: 内部计算时结果值不在目标类型的范围内
- (2) `overflow_error`: 算数运算时发生上溢
- (3) `underflow_error`: 算数运算时发生下溢

21.3.3 自定义异常类

除了使用内置类型和标准异常之外，我们还可以定义自定义异常类。虽然 C++ 对自定义异常类没有特殊限制，但在实践中，通常建议从标准异常类派生出自定义异常类。最常见的做法是继承 `std::exception`、`std::logic_error` 或 `std::runtime_error` 这三个标准异常类。下面来看看 `std::exception` 类的部分成员：

```

1  class exception {
2  public:
3      exception( ) noexcept;
4      explicit exception(char const * const message) noexcept;
5      exception(const exception & rhs) noexcept;
6      exception & operator=(const exception & rhs) noexcept
7      virtual ~exception( ) noexcept;
8  public:
9      virtual const char * what( ) const;
10 };

```

自定义异常类时也应遵循 `std::exception` 类的规范要求。例如：

```

1  #include <iostream>
2  #include <string>
3  #include <stdexcept>
4  class MyException : public std::exception {
5  public:
6      MyException( ) noexcept = default;
7      explicit MyException( int id ) noexcept
8          :std::exception( ( std::string( "我的自定义异常:" ) + std::to_string( id ) ).c_str
9          ( ) ), mErrorID( id )
10         { }
11      MyException( const MyException & rhs ) noexcept = default;
12      MyException & operator=( const MyException & rhs ) noexcept = default;
13      virtual ~MyException( ) noexcept = default;
14      int errorID( ) const { return mErrorID; }
15  private:

```

```

16     const int    mErrorID = 0;
17 };
18 int main( ) {
19     try {
20         throw MyException( 101 ); //手动抛出自定义异常
21     }
22     catch ( const MyException & e ) {
23         std::cout << e.what( ) << std::endl;
24         std::cout << e.errorID( ) << std::endl;
25     }
26 }

```

21.4 异常规范

异常规范也称异常说明,用于描述函数在执行过程中是否会产生异常。从 C++98、C++11、C++14 到 C++17 标准,对于异常规范的语法格式不断演进和变化,语义也有所不同。这里按 C++17 的标准说明异常规范,其语法格式为。

格式 1: *noexcept*

格式 2: *noexcept* (布尔表达式)

其中形式 1 是形式 2 的特例情况,等价于 *noexcept(true)*。形式 2 中的布尔表达式必须是编译期可计算的,表达式中可以使用逻辑运算符和 *noexcept* 运算符。当布尔表达式的计算值为真时,表示函数不产生未处理的异常;当计算值为假时,表示函数可能会产生未处理的异常。例如:

```

1  #include <iostream>
2  class B {
3  public:
4      static const bool CONDITION = true;
5      void func1( ) noexcept( CONDITION ) { }
6      void func2( ) { }
7  private:
8      int  mNum = 1;
9  };
10 class A {
11 public:
12     void f( ) noexcept { }
13     int  g( ) { return 1; }
14     int  h( ) noexcept( false ) { return 2; }
15     void k( ) noexcept( noexcept(B::B( )) && noexcept(B( ).func1( )) && noexcept(B( ).
func2( ))) {
16         B b;

```

```

17         b.func1( );
18         b.func2( );
19     }
20 };
21 int main( ) {
22     A a;
23     B b;
24     std::cout << std::boolalpha <<
25         << "noexcept(a::f( )) = " << noexcept( a.f( )) << std::endl //true
26         << "noexcept(a::g( )) = " << noexcept( a.g( )) << std::endl //false
27         << "noexcept(a::h( )) = " << noexcept( a.h( )) << std::endl //false
28         << "noexcept(a::k( )) = " << noexcept( a.k( )) << std::endl //false
29         << "noexcept(b::func1( )) = " << noexcept( b.func1( )) << std::endl //true
30         << "noexcept(b::func2( )) = " << noexcept( b.func2( )) << std::endl; //false
31 }

```

对于没有显式指定异常规范的函数，默认等价于 `noexcept(false)`，这意味着该函数在执行过程中可能会抛出异常，例如 `B::func2()`。而 `noexcept(true)` 则表明函数不会抛出任何未经处理的异常，如 `A::f()`。需要特别注意的是，如果在声明为 `noexcept(true)` 的函数中抛出了未处理的异常，这表示程序代码违反了函数调用的约定，此时程序将直接调用 `std::terminate()` 终止运行，而不会继续执行后续的异常处理流程。

对于普通函数，可以通过异常规范指明是否可能抛出异常，但构造函数和析构函数需要特别注意。

在构造函数的执行过程中，如果抛出了异常，表明对象未成功构造完成。此时，系统会自动回收已分配的栈内存，但如果在构造函数中动态分配了堆内存，并且后续操作抛出异常，则可能导致内存泄漏。因此，虽然构造函数可以抛出异常，但必须确保不会因异常而出现资源安全问题。这可以通过资源获取即初始化(RAII)技术来避免。

对于析构函数，如果在其执行过程中抛出异常，将会中断栈展开过程，导致程序调用 `std::terminate` 函数并终止运行。因此，必须确保所有析构函数都不会抛出异常，即析构函数应声明为无异常抛出，如类型 `T` 的析构函数最理想的原型是：`T::~T() noexcept;`

请看下面的例子代码：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4  struct B {
5      B( int a, int b ) :x( a ), y( b ) { }

```



```

6     int x;
7     int y;
8 };
9 class A {
10 public:
11     A( int x, int y ) :mpB( new B(x,y) ) {
12         raiseE( );
13     }
14     ~A( ) noexcept { delete mpB; }
15     void f( ) {
16         cout << mpB->x << "," << mpB->y << endl;
17     }
18     void raiseE( ) { throw 88; }
19 private:
20     B * mpB;
21 };
22 int main( ) {
23     try {
24         A aA( 1, 2 );
25         aA.f( );
26     } catch (...) {
27         cout << "出现异常" << endl;
28     }
29 }

```

在类 A 的构造函数中,首先在堆区创建了 B 类对象,随后调用 raiseE 函数并抛出异常。在 main 函数中,try 语句块捕获到异常后会终止当前语句块的执行,因为 aA 对象尚未完全构造,所以其析构函数也不会自动调用。最终,异常由 catch 语句捕获并输出相应的字符串。然而,在这个过程中,堆区中创建的 B 类对象并未得到释放,从而导致内存泄漏。作为对比,如果 A::A()这样定义:

```

1 class A {
2 public:
3     A( int x, int y ) {
4         std::unique_ptr<B> b = std::make_unique<B>( x, y );
5         raiseE( );
6         //将指针赋给 mpB, b 中的指针重置为 nullptr,release( )是无异常的
7         mpB = b.release( );
8     }
9     //其它, 略
10 };

```

当 raiseE() 触发异常时,此时栈空间中已存在构建完成的对象 b。在构造函数退出时,对象

b 的析构函数会被自动调用，从而释放其在堆区分配的内存空间，确保不会发生内存泄漏。

21.5 异常处理

21.5.1 异常处理模型

程序执行中出现了异常或错误时，可以针对异常做相应的处理，不同语言对异常的处理方式可以是以不同的，这些异常处理模型主要分为恢复模型和终止模型。例如下面是 Visual Basic 中采用异常恢复模型的代码片段：

```
1 Public Sub InitializeMatrix(ByVal Var1 As Object, ByVal Var2 As Object)
2     On Error GoTo ErrorHandler
3     ' Insert code that might generate an error here
4     Exit Sub
5 ErrorHandler:
6     ' Insert code to handle the error here
7     Resume Next
8 End Sub
```

代码中，当第 3 行中的语句产生异常时，跳转到第 5 行后，执行第 6 行中的代码，然后执行第 7 行的 `resume next`，即请求程序的执行流恢复到第 3 行中产生异常的语句，继续执行。由于异常的特点，不能保证经过处理后总能恢复程序的执行过程，即使能恢复执行，执行的结果也可能是无意义的。因此，虽然 Visual Basic 语言提供了恢复模型和终止模型两种可选的处理方式，但推荐使用终止模型。相对于恢复模型，使用终止模型时，产生异常后，若提供了异常处理过程，则转处理程序；若没有提供，程序将通过栈展开(`unwind`)进行回滚。在终止模型中，无论是否提供异常处理过程，程序都不再尝试恢复执行程序原有的执行流，即执行流不会返回到异常的产生点，因此称为终止模型。C++语言中只支持异常处理的终止模型。

看如下的代码：

```
1 A * pA = new A( 1 );           //产生 std::bad_alloc 或其它类型的异常
2 if ( !pTemp->doSomeWork( ) ) //此语句也可能产生异常。
3     throw EDoWork( "do some work failed!" ); //手动抛出自定义异常
4 delete pA;
```

由于采用的是终止模型，若第 1 行产生异常，那么构造失败，虽不会执行类 A 的析构函数，但当前栈内的局部对象会自动释放，同时释放堆空间中为 A 类对象分配的内存，但创建过程中分配的其它堆空间不会被自动释放，可能会导致内存泄漏。当第 2 行或第 3 行产生异常时，动态内存一

定已分配，但程序不会执行第 4 行 `delete pA;` 语句，必将导致内存泄漏。从中也可以看出，当存在异常情况时，程序的执行流会有多种可能，要确保程序执行具有正确的处理逻辑，需要在异常产生时，能够捕获异常并进行必要的处理工作。

21.5.2 异常的捕获和处理

在 C++ 中要捕获并处理异常，可使用 `try-catch` 语法结构。其语法格式为：

```
try [ : 类的初始化列表 ] {
    保护语句块
} catch( 类型 1 异常对象 ) {
    处理语句块 1
} [ catch( 类型 2 异常对象 ) {
    处理语句块 2
} ] [ catch( ... ) {
    处理语句块 n
} ]
```

其中的 `catch` 块可以多个，最后一个 `catch(...)` 是可选的，表示匹配任何类型的异常。`try-catch` 结构的执行过程如下：

- (1) 执行类的初始化列表和保护语句块，若没有异常产生，转(5)；若产生了异常，终止保护语句块的执行，清理保护语句块中的自动量，转(2)。
- (2) 查找下一个异常类型匹配的 `catch` 语句，若找到，转(3)；若没有匹配的，转(4)。
- (3) 执行相应的处理语句块，执行完毕后，转(5)。
- (4) 若 `catch` 语句都不匹配，终止代码块所在函数的执行，进行堆栈展开，即执行必要的清栈操作后，将未处理的异常向调用的上一层抛出，转(6)。
- (5) 执行 `try-catch` 块之后的语句，直至所在函数结束，转(6)。
- (6) 结束本函数并返回上一层。若退出 `main` 函数时，仍有未处理的异常，直接终止程序执行，即调用 `std::terminate()` 函数，期间会调用 `abort` 系统函数终止程序。

分析上述执行过程可以看出，如果在语句块执行的过程中产生了异常，由 `catch` 语句捕获并处理。如果没有匹配的 `catch` 语句，则终止本函数的执行，同时将该异常向上一层抛出。如果找到匹配的 `catch` 语句，执行相应的处理过程，在此过程中，如果没有新的异常产生，相当于最初的语句块没有产生异常，则转到 `try-catch` 之后的语句继续执行；如果处理过程又产生了新异常，则终止本函数的执行，同时将该新异常向上一层抛出。例如：

```
1  #include <iostream>
```

```

2  #include <stdexcept>
3  using namespace std;
4  class B {
5  public:
6      B( int n ) { }
7  };
8
9  class A {
10 public:
11     A( int n ) try : mpB( new B( n ) ) {
12         //... 构造函数的函数体
13         cout << "ctr is ok\n";
14     } catch (std::bad_alloc &) {
15     }
16     void f( int n ) {
17         try {
18             cout << "g(n) =" << g( n ) << endl;;
19         } catch (int e) {
20             cout << "Error = " << e << endl;
21         } catch (const std::invalid_argument & e) {
22             cout << "Invalid Argument = " << e.what( ) << endl;
23         } catch (const std::logic_error & e) {
24             cout << "Logic Error = " << e.what( ) << endl;
25         }
26     }
27     int g( int n ) {
28         if (n > 100 || n < 0) throw std::logic_error( "xxxx" );
29         if (n % 3 == 1) throw std::invalid_argument( "yyyyy" );
30         else if (n % 3 == 2) throw 2;
31         return n / 3 + 1;
32     }
33 private:
34     B * mpB;
35 };
36 int main( ) {
37     A aA(1);    //输出 ctr is ok
38     aA.f(33 );  //输出 g(n) = 12
39     aA.f(35);   //输出 Error = 2
40     aA.f(34 );  //输出 Invalid Argument = yyyyy
41     aA.f(1000 ); //输出 Logic Error = xxxx
42 }

```

特别强调，在匹配 `catch` 语句的过程中，严格按照 `catch` 的定义顺序依次检查，因此 `catch` 的顺序应按子类、父类、祖先类的顺序捕获。下面给出一个反例：

```

1  class EMy: public std::exception {
2  public:
3      explicit EMy(char const * const message) noexcept
4          : std::exception(message) { }
5  };
6  class EMyExt:public EMy {
7  public:
8      explicit EMyExt(char const * const message) noexcept
9          : EMy(message) { }
10 };
11 class A {
12 public:
13     void f( int n) {
14         try {
15             if( n > 100 )throw EMyExt("参数大于 100");
16             else if( n < 0) throw EMy("参数小于 0");
17             else doSome( );
18         } catch( EMy & e) {
19             //略
20         } catch( EMyExt & e) {
21             //略
22         }
23         //略
24     }
25 };

```

在示例中，由于先捕获父类型异常再捕获子类型异常，无论在 try 代码块中抛出的是 EMy 类型异常还是 EMyExt 类型异常，都会被第一个匹配的 catch(EMy &e) 捕获，而后续的 catch(EMyExt &e) 永远不会被触发。

21.5.3 异常的再抛出

当使用 catch 语句捕获到异常时，不仅可以进行相应的处理，还可以选择将异常继续向上层抛出，这被称为异常的再抛出。例如：

```

1  void f( ) {
2      try {
3          doSome( );
4      } catch( MyE & e) {
5          cout << "f( )中触发 MyE 类型异常." <<endl;
6          throw;
7      }
8  }

```

如果 `doSome()` 函数抛出的异常类型为 `MyE` 或其派生类, 则该异常将被捕获, 并执行输出字符串的操作。之后, 异常会通过 `throw` 语句继续向上层抛出, 这相当于在 `catch` 语句块中重新抛出异常, 新抛出的异常可以是捕获的原始异常, 也可以是原始异常的拷贝副本, 甚至是全新类型的异常。

需要注意的是, `throw;` 和 `throw e;` 的语义有所不同。`throw;` 表示直接将捕获的原始异常继续向上层抛出, 而 `throw e;` 则表示通过拷贝构造创建一个新的 `MyE` 类型异常对象, 并将这个新对象向上层抛出。

21.5.4 异常的传播和栈展开

函数可能抛出异常, 但是否直接在函数中捕获并处理该异常是可选的。如果在产生异常的函数中没有捕获和处理异常, 那么该异常将自动向上一层抛出。这里的“上一层”可以是嵌套结构中的外层代码块, 也可以是调用链中的上级函数。如果在这一层仍然没有处理该异常, 则继续向上抛出, 重复这一过程, 直至传播到最顶层的 `main` 函数。若 `main` 函数也未捕获并处理该异常, 将自动调用 `std::terminate()`, 从而终止整个程序的执行。这个过程称为异常传播(exception propagation)。

在异常处理和传播中, 总是伴随着栈展开(unwind)的过程, 栈展开负责管理异常情况下栈空间的清理和回收。具体来说, 当出现异常时, 立即清理当前栈, 释放当前栈中的所有本地对象和参数对象。之后, 如果异常继续向上一层抛出, 则由上一层继续栈展开过程。

栈展开只对栈空间负责, 不保证堆区内存的安全, 因此, 在存在异常的情况下, 使用堆区内存需要格外小心。

21.5.5 嵌套的异常处理

异常处理也可以是嵌套的。如果多条语句都可能产生异常, 并且各语句的异常处理过程是不同的, 那么就需要嵌套的异常处理。例如:

```
1 void f( ) {
2     try {
3         int result = g( );
4         try {
5             h( result );
6         } catch ( std::exception & e ) {
7             cout << e.what( ) << endl;
8         }
9     } catch ( int e ) {
10        cout << "e = " << e << endl;
11    } catch ( std::logic_error & e ) {
12        cout << e.what( ) << endl;
```

```

13     }
14 }

```

这里 `g( )` 和 `h(result)` 是两条顺序执行的语句，但每条语句都可能抛出异常。通过嵌套异常处理可以应对各种可能的执行路径。然而，从这个例子可以看出，嵌套的异常处理会使代码变得复杂且难以理解，因此通常不建议使用嵌套的异常处理。作为替代方案，可以将多条语句抽象并封装成一个函数，使用单层的 `try-catch` 结构来捕获该函数中抛出的不同类型的异常。例如，可以重新实现为：

```

1  void k( ) {
2      int result = g( );
3      try {
4          h( result );
5      } catch(...) {
6          throw EH("h( )异常");
7      }
8  }
9  void f( ) {
10     try {
11         k( );
12     } catch ( int e ) {
13         cout << "e = " << e << endl;
14     } catch ( EH & e ) {
15         cout << e.what( ) << endl;
16     } catch ( std::logic_error & e ) {
17         cout << e.what( ) << endl;
18     }
19 }

```

21.5.6 异常的最佳实践

在异常处理过程中，建议遵循以下原则以确保代码的健壮性和可维护性：

1. **合理定义捕获顺序：**当使用多个 `catch` 语句捕获异常时，应按照从具体到抽象的顺序定义捕获次序，即先捕获内置类型、子类异常，再捕获父类和祖先类异常，这有助于避免隐藏特定异常类型的处理逻辑。
2. **生成对象型异常，按引用捕获异常：**当 `try` 块抛出异常对象时，无论是标准异常还是自定义异常，异常对象通常是一个值对象，以便确保异常对象在离开 `try` 块时能正确析构，因此 `catch` 语句捕获的异常一定是经拷贝构造而来的新对象，为了避免不必要的拷贝构造，建议使用引用形式(如 `catch (const ExceptionType & e)` 捕获异常对象，以提高效率并保持

对象的一致性。

3. **在构造函数中使用 RAII(资源获取即初始化)技术来分配堆区资源。**构造函数是可能抛出异常的，必须确保在构造过程中如果发生异常，已分配的资源能够得到正确释放。为此，可以通过 RAII 技术，在构造函数中利用栈上对象管理堆区资源。这样，一旦抛出异常，栈上对象的析构函数会自动触发，从而安全地释放之前分配的堆区资源。
4. **防止析构函数和资源释放函数抛出异常：**自定义的析构函数和其他资源释放函数(如 `operator delete`)应遵守 `noexcept(true)` 规范，确保在释放资源时不抛出未处理的异常，从而避免程序因异常而意外终止。具体来说，在实现这些函数时，应避免调用可能抛出异常的函数。
5. **减少全局变量的使用：**由于异常处理机制中的栈展开仅对栈上的局部变量有效，如果函数访问全局变量，则确保异常安全性会变得更加复杂。因此，尽量减少全局变量的使用，以简化异常处理逻辑。
6. **只捕获能够处理的异常：**对于无法处理的异常，不应在当前层进行捕获，而应让其传递给上层处理。直接终止程序也比吞掉异常更为安全，因此应谨慎使用 `catch(...)` 捕获所有异常。
7. **彻底处理捕获的异常：**虽然 C++ 允许在异常处理中部分处理异常并通过 `throw;` 再次抛出，但这可能导致程序逻辑难以跟踪，容易引入更多的潜在缺陷。因此，建议在捕获异常后尽可能彻底地处理它，避免使用 `throw;` 进行异常再抛出。

21.5.7 关于 **finally**

在其它面向对象语言的异常处理中，通常支持 **finally** 关键字。例如下面 Java 语言中的异常处理：

```

1 public class ExceptionHandlingExample {
2     public static void main(String[] args) {
3         try {
4             // 可能会抛出异常的代码
5             int result = divide(10, 0);
6             System.out.println("结果是: " + result);
7         } catch (ArithmeticException e) {
8             // 捕获并处理异常
9             System.out.println("发生了算术异常: " + e.getMessage());
10        } finally {
11            // 无论是否发生异常，都会执行的代码

```



```

12         System.out.println("这是 finally 块，总是会执行。");
13     }
14 }
15 // 一个可能会抛出异常的方法
16 public static int divide(int a, int b) {
17     return a / b;
18 }
19 }

```

运行结果：

```

发生了算术异常：/ by zero
这是 finally 块，总是会执行。

```

finally 块的含义是在执行 **try** 块之后，无论是否抛出异常，最终都会执行 **finally** 块中的代码。这表明 **finally** 块提供了一个在函数返回前执行清理操作的机会，无论异常是否发生。对此，目前存在两种观点：一种认为 **finally** 块有必要，可以使代码结构更清晰，并有助于提高程序的可靠性；另一种以 C++ 为代表的语言则认为没有必要引入 **finally** 关键字，因为 C++ 语言本身提供了更为安全合理的清理机制。

虽然 C++ 标准中并未引入 **finally**，但部分编译器厂商提供了变形的 **finally** 实现。C++ 标准委员会认为，C++ 的异常处理机制已经支持栈展开，参数对象和局部对象会自动析构，而堆区对象可以通过资源获取即初始化(RAII)技术进行更可靠的资源管理，无需依赖程序员手动编写清理代码。总之，C++ 认为 RAII 和析构函数已经提供了高效且可靠的资源管理方式，更符合其设计哲学，没有必要引入 **finally**。

21.6 异常的安全性

21.6.1 异常安全

在定义函数时，对于可能产生的异常，除了可以提供异常处理之外，还应该针对异常对程序状态和安全性的影响提供额外的保证，也称异常的安全性保证。通常认为有四个级别：

1. **无保证**。如果函数抛出异常，则程序可能不处于有效状态：可能已发生资源泄漏、内存损坏或其他破坏不变性的情况。这种是最弱的异常保证。
2. **基本异常的安全保证**。如果函数抛出未处理异常，则程序仍处于有效状态：没有资源泄露，所有对象的不变式仍满足。
3. **强异常的安全保证**。如果函数抛出未处理异常，将导致回滚的发生，同时确保程序状态返回到函数调用前的状态。

4. **无异常的安全保证。**一个函数具有 `noexcept(true)` 的异常规范, 那么该函数在执行期不应产生未处理的异常, 一旦产生未处理异常将直接终止程序, 例如缺省析构函数, `std::swap` 函数等。这种是最严格的异常安全保证。

一个函数的异常安全等级是由其调用的函数中异常安全等级最低的函数决定的, 这也意味着在一个类中, 即使只有一个成员函数是无保证的安全等级, 而其它成员函数都是无异常安全保证的, 那么整个类也是无保证的安全等级。在这几种异常安全保证中, 无保证是等级最低的, 实现最简单, 但它是危险的, 应该避免在程序中使用; 无异常安全保证是最严格的, 等级最高, 但只有一些特殊的函数才能满足, 绝大多数函数是无法做到的; 强异常安全保证和基本异常安全保证是通常采用的。异常安全性本质上是要求满足两个方面的条件:

- 不发生资源泄漏。要保证所有申请的资源, 如内存、句柄、锁等, 即使发生了异常, 也会被正确释放。
- 不产生无效的数据。在函数异常发生后, 强异常安全保证要求, 所有对象必须与函数调用前一致, 这当然不会产生无效的对象和数据; 基本异常安全保证要求所有的对象必须仍满足不变性要求, 即所有对象必须仍是合法对象, 但不保证所有对象仍有效。例如, 一个 `Person` 对象的不变性可能要求其 `age` 数据成员必须大于等于 0, 执行 `work` 行为时, 由于异常的产生, 其 `age` 从 18 岁变更为 0, 这时的 `Person` 对象仍满足不变式要求, 仍是合法对象, 但其有效性难以保证。再比如, 一个对象中含有一个指针成员 `mPtr`, 不变性要求 `mPtr` 只能为 `nullptr` 或非空指针, 由于异常产生, 若 `mPtr` 成为悬浮指针, 那么此时的对象就是一个合法对象, 但不是一个有效对象。

21.6.2 解决方案

要防止资源泄漏, 可以采用 `RAII(Resource Acquisition Is Initialization)` 技术, 也称资源获取即初始化。例如, 下面的代码可能产生资源泄漏:

```
1  #include <mutex>
2  void f( ) {
3      std::mutex mtx;
4      mtx.lock( );
5      doSomething( ); //若产生异常, 下条语句不会执行, 锁不会被解锁
6      mtx.unlock( );
7  }
8  void g( ) {
9      A * pA = new A( 100 );
```

```

10     doSomething( ); //若产生异常，下条语句不会执行，已分配的内存不会被释放
11     delete pA;
12 }

```

采用 RAII 技术，可改写为：

```

1  #include <mutex>
2  #include <memory>
3  std::mutex mtx; //全局互斥量
4  void f( ) {
5      std::lock_guard<std::mutex> lock( mtx ); //自动锁
6      doSomething( ); //若产生异常，lock 自动析构，正确处理互斥量
7  }
8  void g( ) {
9      std::shared_ptr<A> spA = std::make_shared<A>( 100 );
10     doSomething( ); //若产生异常，spA 自动析构，释放已分配的内存
11 }

```

强异常安全保证和基本异常安全保证都要防止无效数据的产生，但在实践中，强异常安全保证的实现更容易一些。通常可采用如下技术：

- (1) 使用写时复制 (Copy and Swap) 技术。当需要修改目标对象时，先在栈区创建目标对象的副本，然后在副本对象上进行修改，若没有异常产生，再通过具有无异常安全保证的函数，如 swap，交换栈区对象和目标对象的数据；若有异常发生，自动析构栈区的副本对象，而原始的目标对象不变。
- (2) 分离修改和读取的功能。当一个函数既要修改数据，又要返回值对象时，函数结束前要执行拷贝构造函数，此时若拷贝产生异常，由于之前的所有修改仍然起作用，将导致函数不满足强异常安全保证。为此，可将函数拆分成两个函数：具有修改功能的函数只负责修改，不负责返回内容；具有读取功能的函数，则负责读取并返回内容，但不修改数据。

例如：

```

1  class Item {
2  public:
3      Item( int n = 0 ) : mpNum( new int( n ) ) { }
4      ~Item( ) noexcept { delete mpNum; }
5  public:
6      void setNum( int n ) { *mpNum = n; }
7      int  getNum( ) const { return *mpNum; }
8  private:
9      int * mpNum = nullptr;
10 };

```

Item 类中各成员函数是满足强异常安全保证的，但 Item 的拷贝构造和赋值函数是缺省的"浅"拷贝和"浅"赋值，这使得使用 Item 类对象会有释放不安全的问题。可以考虑使用 RAII 技术。如：

```

1  class Item {
2  public:
3      Item( int n = 0 ) : mpNum( std::make_shared<int>( n ) ) { }
4  public:
5      void setNum( int n ) { *mpNum = n; }
6      int  getNum( ) const { return *mpNum; }
7  private:
8      std::shared_ptr<int> mpNum;
9  };

```

此时的 Item 不需要显式自定义析构、拷贝和赋值函数，由于使用了共享指针，指针的释放变安全了，代码也更简洁而且仍具有强异常安全保证。如果希望 Item 类能够"深"拷贝和"深"赋值，可以再使用写时复制(Copy on Write)和复制再交换(Copy and Swap)技术。如：

```

1  class Item {
2  public:
3      Item( int n = 0 ) : mpNum( std::make_shared<int>( n ) ) { }
4      Item( const Item & rhs ) = default;
5      Item & operator=( Item rhs ) {
6          swap( mpNum, rhs.mpNum );
7          return *this;
8      }
9  public:
10     void setNum( int n ) {
11         if (mpNum.use_count( ) > 1)
12             mpNum = make_shared<int>( *mpNum );
13         *mpNum = n;
14     }
15     int  getNum( ) const { return *mpNum; }
16 private:
17     std::shared_ptr<int> mpNum;
18 };

```

再例如，定义一个存放 Item 的栈结构：

```

1  class Stack {
2      static const int CAPACITY = 10;
3  public:
4      Stack( ) = default;
5      Stack( const Stack & ) = delete;

```

```

6      Stack & operator=( const Stack & ) = delete;
7      ~Stack( ) { while ( !empty( ) ) pop( );}
8  public:
9      bool empty( ) const noexcept { return mTopIndex == 0; }
10     void push( const Item & item ) {
11         if ( mTopIndex + 1 >= CAPACITY ) throw "Stack is Full.";
12         Item * p = new Item( item );
13         mpItems[ ++mTopIndex ] = p;
14     }
15     Item pop( ) {
16         if ( empty( ) ) throw "Stack is Empty.";
17         Item result( *mpItems[ mTopIndex ] );
18         delete mpItems[ mTopIndex ];
19         --mTopIndex;
20         return result;
21     }
22 private:
23     int      mTopIndex = 0;
24     Item *    mpItems[ CAPACITY ] { };
25 };

```

在不考虑边界检查的情况下，这里的 `push` 函数是强异常安全的，但对于 `pop` 函数，则不具有强异常安全性保证。其原因是：若 `return result;` 语句失败，即拷贝构造返回的 `Item` 对象失败，那么 `Stack` 将不再是调用 `pop()` 之前的状态。与此同时，类的析构函数应该具有 `noexcept(true)` 异常规范，而 `Stack` 的析构函数由于调用了 `pop` 函数，不具有 `noexcept(true)` 异常规范。下面，通过分离修改和读取功能的方式，修改 `Stack` 的定义，如下：

```

1  class Stack {
2      static const int CAPACITY = 10;
3  public:
4      Stack( ) { }
5      Stack( const Stack & ) = delete;
6      Stack & operator=( const Stack & ) = delete;
7      ~Stack( ) noexcept { //新添加了 noexcept(true) 异常规范
8          while ( !empty( ) ) pop( );
9      }
10 public:
11     bool empty( ) const noexcept { return mTopIndex == 0; }
12     void push( const Item & item ) {
13         if ( mTopIndex + 1 >= CAPACITY ) throw "Stack is Full.";
14         Item * p = new Item( item );
15         mpItems[ ++mTopIndex ] = p;

```

```

16     }
17     Item top( ) const {
18         if ( empty( ) ) throw "Stack is Empty.";
19         return Item( *mpItems[ mTopIndex ] );
20     }
21     void pop( ) noexcept { //等价于 noexcept(true) 异常规范
22         if ( mTopIndex <= 0 || mTopIndex >= CAPACITY ) return;
23         delete mpItems[ mTopIndex ];
24         --mTopIndex;
25     }
26 private:
27     int      mTopIndex = 0;
28     Item *    mpItems[ CAPACITY ] { };
29 };

```

将原 pop() 分成 top() 和 pop() 两个函数，分别负责获取栈顶元素和出栈。通过分离，pop 函数甚至具有了无异常的安全保证，析构函数也具有了 noexcept(true) 异常规范，top() 函数则是强异常安全的。

21.7 std::optional 和 std::expected

错误码和异常机制各具优势。错误码因其高效性，常被应用于对系统性能要求高的场景，例如嵌入式程序和程序库；而异常机制在处理复杂的异常逻辑时更具优势。C++ 新标准引入的 std::optional 和 std::expected，为程序员提供了更多处理错误和异常的选择。

21.7.1 std::optional

std::optional<T> 是 C++17 引入的一个类模板，用于表示可能包含值或不包含值的对象。它提供了一种类型安全的替代方案，避免了传统上使用特殊值(如 nullptr 或 -1)来表示“无值”状态的做法。当函数可能返回无效值时，可以将 std::optional 用作返回类型，例如：

```

1  #include <iostream>
2  #include <optional>
3  std::optional<int> divide(int a, int b) {
4      if ( b == 0 ) {
5          return std::nullopt; // 表示无值
6      }
7      return a / b;
8  }
9  int main( ) {
10     auto result = divide(10, 0);

```

```

11     if ( result.has_value( ) ) {
12         std::cout << "计算结果: " << result.value( ) << std::endl;
13     } else {
14         std::cout << "除零错误!" << std::endl;
15     }
16     return 0;
17 }

```

`std::optional` 适用于需要区分函数返回的正常值和无效值的情形，尤其当程序逻辑主要围绕处理多种正常值展开时。

21.7.2 `std::expected`

在 C++23 中，标准库引入了 `std::expected<T, E>`，这是一种用于表示操作可能成功或失败的结果类型。`std::expected` 类似于 `std::optional`，但不仅能够表示“有值”或“无值”，还可以携带更详细的错误信息。

当将 `std::expected<T, E>` 用作函数的返回类型时，它可以返回表示成功的 `T` 类型值，或者表示错误的 `E` 类型值。使用 `std::expected<T, E>` 可以替代传统的错误码和异常处理，并且相比异常处理，它通常具有更高的效率。例如：

```

1  #include <iostream>
2  #include <expected>
3  #include <string>
4  std::expected<int, std::string> divide(int a, int b) {
5      if (b == 0) {
6          return std::unexpected("除零!"); // 返回错误
7      }
8      return a / b; // 返回成功值
9  }
10 int main( ) {
11     auto result = divide(10, 0);
12     if (result) {
13         std::cout << "结果: " << *result << std::endl;
14     } else {
15         std::cout << "错误: " << result.error( ) << std::endl;
16     }
17     return 0;
18 }

```

本例中的错误类型为 `std::string`，但也可以是其它类型，包括内置类型、枚举类型、自定义类或结构等。例如：

```

1  enum class ErrorCode {
2      SUCCESS = 0,
3      DIVISION_BY_ZERO = 1,
4      INVALID_INPUT = 2
5  };
6  std::expected<int, ErrorCode> divide(int a, int b) {
7      if (b == 0) {
8          return std::unexpected(ErrorCode::DIVISION_BY_ZERO);
9      }
10     return a / b;
11 }

```

在使用 `std::expected<T, E>` 时，常用的操作有：

- `operator bool`: 返回 `true` 表示成功，`false` 表示失败。
- `has_value()`: 与 `operator bool` 等价。
- `operator*`: 返回成功值。如果失败，行为未定义。
- `error()`: 返回错误值。如果成功，行为未定义。
- `value()`: 返回成功值。如果失败，抛出 `std::bad_expected_access` 异常。
- `value_or(default)`: 链式操作。如果成功则返回值，否则返回 `default` 值。
- `and_then()`: 链式操作。如果成功，继续执行下一个操作。
- `or_else()`: 链式操作。如果失败，处理错误。

例如：

```

1  #include <iostream>
2  #include <expected>
3  #include <string>
4  std::expected<int, std::string> square(int x) {
5      return x * x;
6  }
7  int main( ) {
8      auto result = divide(10, 2)
9          .and_then(square)    // 链式操作
10         .value_or(-1);      // 如果失败，返回默认值
11     std::cout << "Result: " << result << std::endl;
12     return 0;
13 }

```

总之，`std::expected` 是 C++23 引入的一种现代化错误处理机制，结合了错误码和异常处理的优点，在需要明确区分成功和失败场景时，可用来替代传统的错误码和异常处理，提高代码的可读

性和安全性。在实际使用中，最常见的是将错误类型定义为描述更清晰的枚举类型，避免使用原始类型，从而提高效率，此外还可以使用 `and_then()`、`or_else()`和 `value_or(default)`链式操作，进一步简化代码逻辑。但对于复杂的错误处理，特别是错误本身需携带额外信息的，应优先考虑异常机制。